

Modelo de Equilíbrio de Stackelberg com um Único DSO

Formulação completa do problema de otimização bilevel

Trabalho derivado da implementação em Julia (módulo `src/planning/` do projeto `TSO-DSO`), resolvida via decomposição de Benders em `solve_stackelberg!` (`src/planning/benders.jl`).

O modelo reside na **camada de planejamento** e representa o equilíbrio de Stackelberg–Nash entre TSO e DSO descrito na nota PSR (Paper 2 em `THEORY-papers.md`). É um problema **bilevel**: o distribuidor (DSO) é o *líder* que investe em flexibilidade y_{inv} e escolhe um perfil de importação $z[t]$; o sistema de transmissão é o *seguidor* que investe em reforço de corredor para entregar essa importação. A decomposição de Benders decompõe o problema em um mestre (o líder) e dois subproblemas (o oráculo operacional e o seguidor de transmissão).

1 Conjuntos e índices

Símbolo	Significado
$t \in \{1, \dots, T\}$	passos de tempo (horizonte do dia anterior, $T = 24$)
$j \in \{1, \dots, N\}$	barras do alimentador; $j = 1 = \text{raiz}$ (fronteira MEM)
$(i, j) \in \mathcal{B}$	trechos radiais, pai $i \rightarrow$ filho j ($N - 1$ no total)

2 Variáveis de decisão

Líder (mestre):

- $y_{\text{inv}} \in [0, y_{\text{max}}]$ — investimento em flexibilidade (contínuo)
- $z[t] \in [0, y_{\text{inv}}]$ — perfil de importação que o líder assume (a **variável de acoplamento**)

Seguidor (LP de transmissão):

- $x_{\text{inv}} \in [0, x_{\text{inv,max}}]$ — investimento em reforço de corredor
- $x_{\text{op}[t]} \geq 0$ — fluxo operacional entregue pelo corredor

Oráculo (SOCP de bem-estar operacional — formulação `ConvexBranchFlow`):

- $v[j, t], \hat{v}[j, t]$ — magnitude de tensão ao quadrado, verdadeira / cópia de exatidão
- $P[b, t], Q[b, t]$ — fluxo ativo/reactivo do trecho
- $l[b, t] \geq 0$ — corrente do trecho ao quadrado
- $p_{\text{import}[t]}$ — intercâmbio de fronteira de sinal livre (fixado em $z[t]$)
- $q_{\text{import}[t]}$ — fronteira reativa de sinal livre (quando a formulação tem canal reativo)
- variáveis dos dispositivos de cada agregador (potência do A/C, potência postergável, carga/descarga/SOC da bateria)

3 O problema bilevel completo

$$\min_{y_{\text{inv}}, z} c_y \cdot y_{\text{inv}} + \alpha_{\text{op}}(z) + \alpha_x(z)$$

sujeito a:

$$0 \leq z[t] \leq y_{\text{inv}}, \quad 0 \leq y_{\text{inv}} \leq y_{\text{max}}$$

onde as duas funções de custo implícitas são definidas por subproblemas:

3.1 (1) Oráculo de bem-estar operacional — $\alpha_{\text{op}(z)}$

Esta é `solve_planning_oracle!` — uma versão fixada em z do SOCP de bem-estar GLB-CVX da tese (eq. 3.38). Seu **negativo** é o custo de bem-estar do líder:

$$\alpha_{\text{op}(z)} = -\max \left[\underbrace{\sum_j U_{ag_j}}_{\text{utilidade dos agregados}} - \sum_t \lambda_0[t] \cdot p_{\text{import}[t]} \right]$$

sujeito à rede DistFlow completa + restrições dos dispositivos dos agregadores.

Rede (ConvexBranchFlow, tese 3.31–3.45):

$$R_{p,j}[t] = P_{ij} - r_{ij} \cdot l_{ij} - p_{ag_j}[t] - \sum_{m:j \rightarrow m} P_{jm} = 0 \quad (\text{balanço ativo, 3.31})$$

$$R_{q,j}[t] = Q_{ij} - x_{ij} \cdot l_{ij} - q_{ag_j}[t] - \sum_{m:j \rightarrow m} Q_{jm} = 0 \quad (\text{balanço reativo, 3.32})$$

$$v_j = v_i - 2(r_{ij}P_{ij} + x_{ij}Q_{ij}) + (r_{ij}^2 + x_{ij}^2) \cdot l_{ij} \quad (\text{queda de tensão, 3.33})$$

$$l_{ij} \cdot v_i \geq P_{ij}^2 + Q_{ij}^2 \quad (\text{relaxação SOC, 3.39})$$

$$\hat{v}_j = \hat{v}_i - 2[r_{ij}(P_{ij} + r_{ij} \cdot l_{ij}) + x_{ij}(Q_{ij} + x_{ij} \cdot l_{ij})] \quad (\text{cópia de exatidão, 3.43})$$

$$V_{\min}^2 \leq v_j, \hat{v}_j \leq V_{\max}^2 \quad (\text{limites de tensão, 3.45})$$

$$P_{ij}^2 + Q_{ij}^2 \leq S_{\max,ij}^2 \quad (\text{limite de potência aparente, 3.36; apenas trecho de cabeça})$$

Agregadores (tese 3.21–3.23):

$$p_{ag_j}[t] = \sum_{h \in H_j} \left(p_{\text{ch}} - p_{\text{dch}} - p_{\text{pv}} + \sum_d p_{h,d} \right) - P_{\text{dc},j}[t]$$

$$q_{ag_j}[t] = -P_{\text{dc},j}[t] \cdot \tan(\arccos \varphi)$$

além das restrições temporais de cada dispositivo (recursão de SOC 3.6, termostático 3.2–3.3, postergável 3.4–3.5) — reutilizando `contribute!` literalmente de `solve_welfare`.

A costura de acoplamento de Benders (D-01):

$$p_{\text{import}[t]} = z[t] \quad (\text{o pino cujo dual } \pi[t] \text{ é o gradiente do corte de Benders})$$

Portanto, $\alpha_{\text{op}(z)}$ é o **bem-estar ótimo negado** quando a importação de fronteira é forçada a igualar o z escolhido pelo líder. Seu gradiente $\nabla \alpha_{\text{op}(z)} = \pi[t]$ vem do dual da restrição de pino.

3.2 (2) Seguidor de reforço de transmissão — $\alpha_{x(z)}$

Esta é `solve_follower!` (`follower.jl`) — um pequeno LP:

$$\alpha_{x(z)} = \min_{x_{\text{inv}}, x_{\text{op}}} c_{\text{inv}} \cdot x_{\text{inv}} + \sum_t c_{\text{op}[t]} \cdot x_{\text{op}[t]}$$

sujeito a:

$$x_{\text{op}[t]} \leq \text{corridor_cap} \cdot x_{\text{inv}} \quad (\text{capacidade por unidade de investimento})$$

$$x_{\text{op}[t]} = z[t] \quad (\text{acoplamento: entregar a importação do líder})$$

$$0 \leq x_{\text{inv}} \leq x_{\text{inv,max}}, \quad x_{\text{op}[t]} \geq 0$$

Seu dual de acoplamento $\pi_{s[t]} = \text{dual de } x_{\text{op}[t]} = z[t] = \text{custo marginal de reforço}$ de uma unidade a mais de importação. Se z for **inentregável** (nenhum reforço pode carregá-lo), o seguidor retorna um **certificado de Farkas** (v, u) em vez disso $\rightarrow$ um corte de factibilidade no mestre.

4 Como Benders resolve o problema (o loop `solve_stackelberg!`)

Como $\alpha_{\text{op}(z)}$ e $\alpha_x(z)$ são funções de valor convexas (mínimo de LP/SOCP), Benders as substitui por **variáveis de epígrafo + cortes** e resolve uma sequência de LPs mestres:

4.1 LP Mestre (construído uma vez, cortes acrescentados — `master.jl`)

$$\min_{y_{\text{inv}}, z, \alpha_{\text{op}}, \alpha_x} c_y \cdot y_{\text{inv}} + \alpha_{\text{op}} + \alpha_x$$

sujeito a:

$$0 \leq z[t] \leq y_{\text{inv}}, \quad 0 \leq y_{\text{inv}} \leq y_{\text{max}}$$

$$\alpha_{\text{op}} \geq \alpha_{\text{op,lb}}, \quad \alpha_x \geq \alpha_{x,\text{lb}} \quad (\text{limites inferiores finitos — evita infactibilidade dual na 1ª iteração, Pitfall M1})$$

Cortes de otimalidade (acrescentados a cada iteração factível k , um por subproblema — Benders **multi-corte**):

$$\alpha_{\text{op}} \geq -\text{oracle_cost}_k + \sum_t \pi_{k[t]} \cdot (z[t] - z_{k[t]})$$

$$\alpha_x \geq \text{follower_cost}_k + \sum_t \pi_{s,k}[t] \cdot (z[t] - z_{k[t]})$$

Cortes de factibilidade (quando z_k é inentregável, do raio de Farkas do seguidor):

$$v_k + \sum_t u_{k[t]} \cdot (z[t] - z_{k[t]}) \leq 0$$

4.2 A iteração

1. Resolva o mestre $\rightarrow$ obtenha z_k e o LB (objetivo do mestre, um limite inferior).
2. Resolva o seguidor(z_k):
 - Se **INFACTÍVEL** $\rightarrow$ adicione corte de factibilidade, **continue** (nunca atualiza UB, nunca chama o oráculo).
 - Caso contrário $\rightarrow$ obtenha o custo e o gradiente $\pi_{s,k}$.
3. Resolva o oráculo(z_k) $\rightarrow$ obtenha o custo e o gradiente π_k .
Adicione cortes de otimalidade em α_{op} e α_x .
4. Custo verdadeiro = $c_y \cdot y_k + \text{follower_cost} - \text{oracle_cost} \rightarrow$ candidato a UB (incumbente).
5. Intervalo = $(UB - LB) / \max(1, |UB|)$; pare se intervalo $\leq 10^{-6}$.
 - **LB** (objetivo do mestre) sobe por baixo à medida que os cortes se acumulam.
 - **UB** (melhor custo verdadeiro encontrado) desce por cima.
 - Converge para o **incumbente** $(y_{\text{best}}, z_{\text{best}})$ — o iterado que alcançou UB, não o último iterado.

5 Resultado

O equilíbrio que o modelo retorna:

$$(y_{\text{inv}}^*, z^*) = \operatorname{argmin} \left\{ c_y y_{\text{inv}} + \alpha_{\text{op}(z)} + \alpha_{x(z)} \right\}$$

é o **equilíbrio de Stackelberg**: o investimento em flexibilidade e o perfil de importação ótimos do distribuidor, **antecipando** a resposta de reforço do sistema de transmissão. O dual $\pi_{s[t]}$ no ótimo é o **custo marginal de reforço de interconexão** — o sinal de TUST (tarifa de uso do sistema de transmissão).

No caso de múltiplos distribuidores (`coupling.jl`), uma diagonalização de Gauss-Seidel sobre o Stackelberg de cada distribuidor converge para o **equilíbrio de Nash**.

6 Taxonomia dos variantes de planejamento

Este modelo (`solve_stackelberg!`) é UM dos três variantes de planejamento que o repositório implementa hoje — cada um com uma natureza teórico-dos-jogos distinta, apesar de todos compartilharem o rótulo informal “Stackelberg”:

Variante	Função Julia	Natureza teórico-dos-jogos	Teste de certificação
Integrado, decomposto por Benders	<code>solve_stackelberg!</code>	Problema único — NÃO é um jogo de dois níveis genuíno; líder e seguidor compartilham o MESMO objetivo (ver “Honest relabelling” no docstring de <code>solve_stackelberg!</code>)	<code>test_planning_benders.jl</code> , <code>test_planning_goldens.jl</code> (PVAL-02)
Bilevel genuíno	<code>solve_bilevel!</code> / <code>build_bilevel_kkt</code>	Stackelberg genuíno — o seguidor minimiza seu PRÓPRIO custo, divergente da valorização do líder	<code>test_planning_bilevel.jl</code> , <code>test_planning_certificati</code> , <code>test_planning_certificati</code>
GNE de restrição compartilhada / VE	<code>run_nash!</code> / <code>solve_var</code>	Legionário de Nash, generalizado (N jogadores, restrição de capacidade compartilhada); VE = GNE com multiplicador comum — único apenas quando o ótimo conjunto é único (ex.: <code>c_inv</code> assimétrico); no fixture simétrico o conjunto de VEs é o próprio continuum de GNEs	<code>test_planning_nash.jl</code>

`BendersMasterInteger` (investimento binário-expansão, `master_integer.jl`) é utilizável via o kwarg `master= TANTO` na linha (1) (`solve_stackelberg!`, desde a Fase 24) QUANTO na linha (3) (`run_nash!`’s kwarg `integer`, desde a Fase 31) — ver `docs/writeups/stackelberg_vs_psr_nln2.typ` para o mapeamento completo contra a nota PSR. A política de inexatidão SOCP da Fase 30 (`inexact_policy`, `:strict/:reject/:certify_incumbent`) aplica-se às linhas (1) e (3), que chamam

`solve_stackelberg!` (a linha 3 em cada resposta ótima de `run_nash!`, que a repassa sem alteração). NÃO se aplica à linha (2): `solve_bilevel!/build_bilevel_kkt` (`bilevel_kkt.jl`) nunca chamam `solve_stackelberg!`, não têm kwarg `inexact_policy` e aceitam apenas a rede `LinDistFlow` — qualquer formulação SOCP/QP/NLP é rejeitada com `ArgumentError` antes da construção do modelo (o MILP de KKT exige uma rede estritamente afim), de modo que não há relaxação cônica cuja inexatidão precise de política. (Correção da revisão de código da Fase 31, WR-02.)

7 Notas de implementação

- O oráculo reutiliza literalmente os construtores de `solve_welfare` (`contribute!(pf, dots)`, `contribute!(agg, dots)`) — o subproblema da camada de planejamento é a resolução de bem-estar DADP, parametrizada na importação via a restrição de pino.
- Todos os três modelos são **construídos uma vez** e resolvidos novamente via `Parameters / set_parameter_value` do JuMP — nunca reconstruídos dentro do loop (espelha a disciplina de construir-uma-vez da camada ADMM).
- Investimento CONTÍNUO é o padrão (`BendersMaster`, LP/SOCP puro — Benders converge finitamente com o teste de intervalo relativo); `BendersMasterInteger` (`master_integer.jl`, Fase 24/31) está disponível via `master=` para investimento inteiro por expansão binária — ver a Taxonomia acima e `docs/writeups/stackelberg_vs_psr_nln2.typ` para o mapeamento completo.

Referências:

Palacios, J. P. *Tese de Doutorado*, UNSJ/CONICET, 2022, Capítulo 4.

Nota PSR: “Reforços interconexões N1–N2 via Stackelberg + Nash” (Português).

Implementação: `src/planning/` no projeto TSO-DSO (`master.jl`, `follower.jl`, `subproblem.jl`, `benders.jl`).